ORICHUNA.CA

Sistema de Asignación de Cupos y Turnos Vehiculares

Documentación técnica y guía de estudio

Materia	Programación Orientada a Objetos
Frontend	HTML5 · CSS3 · JavaScript ES6+ (POO)
Backend	PHP 8.2 · PDO
Base de datos	MySQL / MariaDB (XAMPP)
Servicios	Apache · MySQL · EmailJS
Versión	2.1 — validación completa y base de datos relacional

Documento de estudio · incluye modelo de datos, API, POO y ejercicios

Índice

1. Qué es el sistema
- 1.1 Módulos
- 1.2 Reglas de negocio

2. Tecnologías y entorno
- 2.1 Por qué se necesita PHP

3. Cómo ejecutar el sistema
- 3.1 Opción A — Con base de datos (recomendado)
- 3.2 Opción B — Sin servidor
- 3.3 Cuentas de demostración
- 3.4 Si algo no arranca

4. Arquitectura
- 4.1 Capas
- 4.2 Árbol de archivos
- 4.3 Recorrido de una acción

5. Modelo de datos
- 5.1 Por qué una base relacional
- 5.2 Diagrama entidad-relación
- 5.3 Tabla usuario
- 5.4 Tabla vehiculo
- 5.5 Tabla ticket
- 5.6 Tabla configuracion
- 5.7 Tabla estado_app
- 5.8 Claves, índices y restricciones
- 5.9 Tipos de datos: decisiones y por qué
- 5.10 Codificación y zona horaria

6. SQL: creación, consultas y mantenimiento
- 6.1 Script de creación (api/schema.sql)
- 6.2 Consultas de estudio
- 6.3 Operaciones de mantenimiento

7. La API en PHP
- 7.1 Endpoints
- 7.2 El contrato JSON
- 7.3 api/config.php — configuración
- 7.4 api/db.php — conexión PDO
- 7.5 api/estado.php — lectura (GET)
- 7.6 api/estado.php — escritura (POST)
- 7.7 api/estado.php — borrado (DELETE)
- 7.8 api/instalar.php — el instalador
- 7.9 Seguridad de la API

8. Cambios en el frontend
- 8.1 La clase Almacenamiento
- 8.2 js/nucleo.js: arranque asíncrono
- 8.3 Flujo completo de arranque
- 8.4 Qué pasa si la API falla

9. Programación orientada a objetos
- 9.1 Validador — métodos estáticos
- 9.1.1 Validar antes de modificar: el error de los números negativos
- 9.2 Usuario y la herencia
- 9.3 Vehiculo, subtipos y el patrón Fábrica
- 9.4 ConfiguracionDistribucion — el día
- 9.5 Ticket — ciclo de vida
- 9.6 EstacionServicio — las reglas
- 9.7 Persistencia y reconstrucción

10. Flujos completos
- 10.1 Login de administrador
- 10.2 Solicitar un cupo (cliente) y despacharlo (personal)
- 10.3 Recargar la página con una solicitud pendiente

11. Guía de estudio
- 11.1 Recorrido recomendado (en este orden)
- 11.2 Preguntas de repaso (con respuesta)
- 11.3 Ejercicios propuestos
- 11.4 Glosario

12. Anexos
- 12.1 Inventario de archivos
- 12.2 Correspondencia entre capas
- 12.3 Lista de verificación para la defensa
- 12.4 Registro de cambios

1. Qué es el sistema

ORICHUNA.CA automatiza la distribución de combustible subsidiado en una estación de servicio. El **cliente** registra su vehículo, elige cuántos litros desea (hasta el máximo de su tipo de vehículo) y **solicita su cupo**; el sistema le asigna un **turno único del 1 al 20**, un **código de verificación único de 3 dígitos** y una **hora de recogida**. El **administrador o empleado** revisa la solicitud en el módulo *Solicitudes de cupo* y **confirma el despacho**, que es cuando se descuenta el combustible. El cliente nunca ve el inventario ni cuántos cupos quedan: solo su turno, placa, litros solicitados, estatus y hora de recogida.

1.1 Módulos

Módulo	Quién lo usa	Qué hace
Inicio	Administrador, Empleado	Resumen del día, modelo de clases y accesos rápidos
Mi cupo de gasolina	Cliente	Solicita el cupo, elige litros y consulta su turno, hora de recogida y estatus
Solicitudes de cupo	Administrador, Empleado	Acepta y confirma el despacho, o cancela con motivo
Tickets	Administrador, Empleado	Tabla con todos los comprobantes
Surtir estación	Administrador, Empleado	Combustible, horario y máximos de litros por tipo de vehículo
Usuarios	Administrador	Alta, edición y baja de cuentas
Reportes	Administrador, Empleado	Litros por tipo/marca/modelo y por estatus
Búsqueda	Administrador, Empleado	Por código de verificación o por placa

1.2 Reglas de negocio

1. **Un cupo por cliente y por día.** El turno es único del 1 al 20; al completarse los 20 se rechazan nuevas solicitudes. Un cliente no puede tener dos solicitudes activas.
2. **Código de verificación único (1–999)** entre *todos* los tickets de la historia.
3. El combustible se descuenta **solo cuando el personal confirma el despacho**. Si no alcanza, la solicitud sigue pendiente y se avisa al personal.
4. **Si no hay litros disponibles**, el cliente ve el aviso "No hay litros disponibles en la estación en este momento" y no puede enviar la solicitud.
5. El cliente elige los litros dentro del **máximo configurado para su tipo de vehículo**. Nunca ve el stock ni cuántos turnos quedan.
6. Fuera del horario configurado no se permiten nuevas solicitudes.
7. Solo las cuentas **verificadas** pueden iniciar sesión.
8. El cliente solo accede al módulo **Mi cupo de gasolina**; aceptar y despachar es exclusivo de administrador/empleado.
9. Todo campo se valida antes de modificar el estado: si un dato es inválido, la operación completa se rechaza y no deja valores a medias. Las mismas reglas se repiten en la API (HTTP 422) y en la base (`UNIQUE`, `ENUM`, `DECIMAL`).

Cambio de la versión 2.0. La versión 1 guardaba la estación completa en `localStorage` del navegador. La versión 2 la guarda en una base de datos **MySQL/MariaDB** mediante una API en **PHP**. El `localStorage` queda como respaldo para que la aplicación siga funcionando sin servidor.

Cambio de la versión 2.1. Se validaron todos los formularios y se corrigió el bug que permitía guardar números negativos en "Surtir estación": ahora se valida sobre un borrador y solo se copia a la configuración si no hay errores. La API también rechaza esos datos con HTTP 422.

Cambio de la versión 2.2. El flujo de gasolina pasó de "el operador asigna y confirma con temporizador" a **"el cliente solicita y el personal despacha"**. Se agregaron la hora de recogida (reparto de los 20 turnos entre la hora de inicio y la de cierre), el identificador de solicitante, la elección de litros dentro del máximo del vehículo y el módulo *Solicitudes de cupo*. El cliente ya no ve inventario, cupos ni temporizador.

2. Tecnologías y entorno

Tecnología	Versión usada	Qué aporta al proyecto
HTML5	—	Estructura: login, secciones de cada módulo y modal de verificación
CSS3	—	Diseño completo: variables, tarjetas, tablas, cronómetro
JavaScript ES6+	—	`class`, `extends`, métodos estáticos, `Set`, `Map`, promesas, `async/await`
PHP	8.2.12	API que recibe y entrega el estado en JSON
PDO	incluido en PHP	Acceso a la base de datos con sentencias preparadas
MySQL / MariaDB	MariaDB 10.4.32	Persistencia relacional (5 tablas)
Apache	2.4 (XAMPP)	Sirve la aplicación y ejecuta el PHP
EmailJS (CDN)	v4	Envío real de códigos de verificación al correo

Nota sobre XAMPP. El paquete XAMPP para Windows incluye **MariaDB**, no MySQL de Oracle. MariaDB es un *fork* compatible: el mismo SQL, el mismo driver y las mismas funciones básicas. En este documento se usa "MySQL" para referirse al motor, y "MariaDB" cuando se menciona la versión concreta de XAMPP.

2.1 Por qué se necesita PHP

Un navegador **no puede abrir una conexión a MySQL** por seguridad: si el JavaScript llevara el usuario y la contraseña de la base de datos, cualquiera podría leerlos con *Ver código fuente* y borrar la base entera.

La solución es una capa intermedia:

```
Navegador (JS)  →  HTTP/JSON  →  Servidor (PHP)  →  PDO  →  MySQL/MariaDB
```

El navegador solo conoce una URL (`api/estado.php`). Las credenciales viven en el servidor (`api/config.php`), fuera del alcance del usuario.

3. Cómo ejecutar el sistema

3.1 Opción A — Con base de datos (recomendado)

1. Copiar la carpeta `orichuna` dentro de `C:\xampp\htdocs\`.
2. Abrir el **Panel de control de XAMPP** y arrancar **Apache** y **MySQL**.
3. Crear la base **una sola vez**: abrir en el navegador `http://localhost/orichuna/api/instalar.php` Debe responder: `OK: base de datos 'orichuna' lista (7 sentencias).`
4. Abrir la aplicación: **`http://localhost/orichuna/`**.

Los datos quedan en la base `orichuna` y sobreviven a cambios de navegador, borrado de caché o reinicio de la PC.

3.2 Opción B — Sin servidor

Abrir `index.html` con doble clic. La aplicación intenta la API, falla y **cae automáticamente en `localStorage`**. Todo funciona igual salvo que los datos quedan atados a ese navegador y a esa PC.

3.3 Cuentas de demostración

Rol	Correo	Teléfono	Permisos
Administrador	`rondonsebastian81@gmail.com`	0412-0000000	Todos los módulos
Empleado	`empleado@orichuna.com`	0424-1112233	Solicitudes, surtir, tickets, reportes, búsqueda
Cliente	se registra en la pestaña "Cliente"	—	Solo mi cupo de gasolina

El código de 4 dígitos **siempre se muestra en el modal** (respaldo) y, si EmailJS responde, también llega al correo real.

3.4 Si algo no arranca

Síntoma	Causa probable	Solución
`Error 2002 Can't connect`	MySQL apagado	Arrancar MySQL en XAMPP
La app carga pero no guarda	API no encontrada (ruta)	Verificar que existe `api/estado.php` y que Apache corre
`Unknown database 'orichuna'`	Falta ejecutar el instalador	Abrir `api/instalar.php`
`Access denied for user 'root'`	Contraseña de MySQL	Ajustar `api/config.php`
Cambios que no se ven	Caché del navegador	`Ctrl + F5`, o subir `?v=` en `index.html`
`Port 80 in use`	Otro programa usa el puerto	Skype/IIS: cambiar el puerto de Apache

4. Arquitectura

4.1 Capas

La aplicación está separada en capas con una responsabilidad cada una:

Capa	Archivos	Responsabilidad	No debe saber
Presentación	`index.html`, `css/estilos.css`, `js/vistas/*.js`	Dibujar y capturar datos	Cómo se guarda nada
Núcleo / aplicación	`js/nucleo.js`	Estado global, login, navegación, modal	SQL
Dominio (POO)	`js/modelos.js`	Reglas de negocio y validaciones	DOM y base de datos
Persistencia	`Almacenamiento` (en `modelos.js`)	Llevar y traer el estado	Reglas de negocio
API	`api/*.php`	Traducir JSON ↔ tablas	Nada de interfaz
Datos	`orichuna` (MySQL)	Guardar y garantizar integridad	Nada de negocio

La regla de oro: **las flechas solo apuntan hacia abajo**. Una vista nunca escribe SQL; la base nunca conoce una clase de JavaScript.

4.2 Árbol de archivos

```
orichuna/
├─ index.html                 → Estructura, pantallas y modal
├─ README.md                  → Guía rápida de uso
├─ Documentacion_ORICHUNA.pdf  → Este documento
├─ css/
│  └─ estilos.css             → Diseño (variables de color, tarjetas, tablas)
├─ js/
│  ├─ modelos.js              → Clases y reglas (POO pura) + Almacenamiento
│  ├─ nucleo.js               → Estado global, login, navegación, EmailJS
│  └─ vistas/
│     ├─ inicio.js            → Resumen y modelo de clases
│     ├─ asignacion.js        → Mi cupo (cliente): solicitud, hora de recogida
│     ├─ solicitudes.js       → Solicitudes: aceptar/despachar o cancelar
│     ├─ tickets.js           → Tabla de comprobantes
│     ├─ estacion.js          → Configuración del día
│     ├─ usuarios.js          → Gestor de usuarios
│     ├─ reportes.js          → Reportes
│     └─ busqueda.js          → Búsqueda por código/placa
└─ api/
   ├─ config.php              → Credenciales de la base de datos
   ├─ db.php                  → Conexión PDO compartida
   ├─ estado.php              → API: GET / POST / DELETE
   ├─ instalar.php            → Crea la base y las tablas
   └─ schema.sql              → Definición SQL de las 5 tablas
```

4.3 Recorrido de una acción

Ejemplo: el personal pulsa **Despachar** en *Solicitudes de cupo*.

```
[1] solicitudes.js       despachar(id)
        ↓ ticket.aceptar(estacion)          ← regla de negocio (POO)
        ↓ guardar()                          ← nucleo.js
[2] nucleo.js            almacenamiento.guardar(estado.estacion)
        ↓ estacion.datosParaGuardar()        ← objeto plano (JSON)
[3] modelos.js           fetch POST api/estado.php
[4] estado.php           lee JSON, abre transacción, escribe 5 tablas
[5] MySQL                confirma la transacción
        ↑ respuesta {"ok":true}
[6] solicitudes.js       pintarSolicitudes()  ← la interfaz ya se actualizó antes
```

La interfaz **no espera** la respuesta del servidor: se pinta al instante y el guardado viaja en segundo plano. Si el servidor falla, el usuario no ve un error y el respaldo local conserva los datos.

5. Modelo de datos

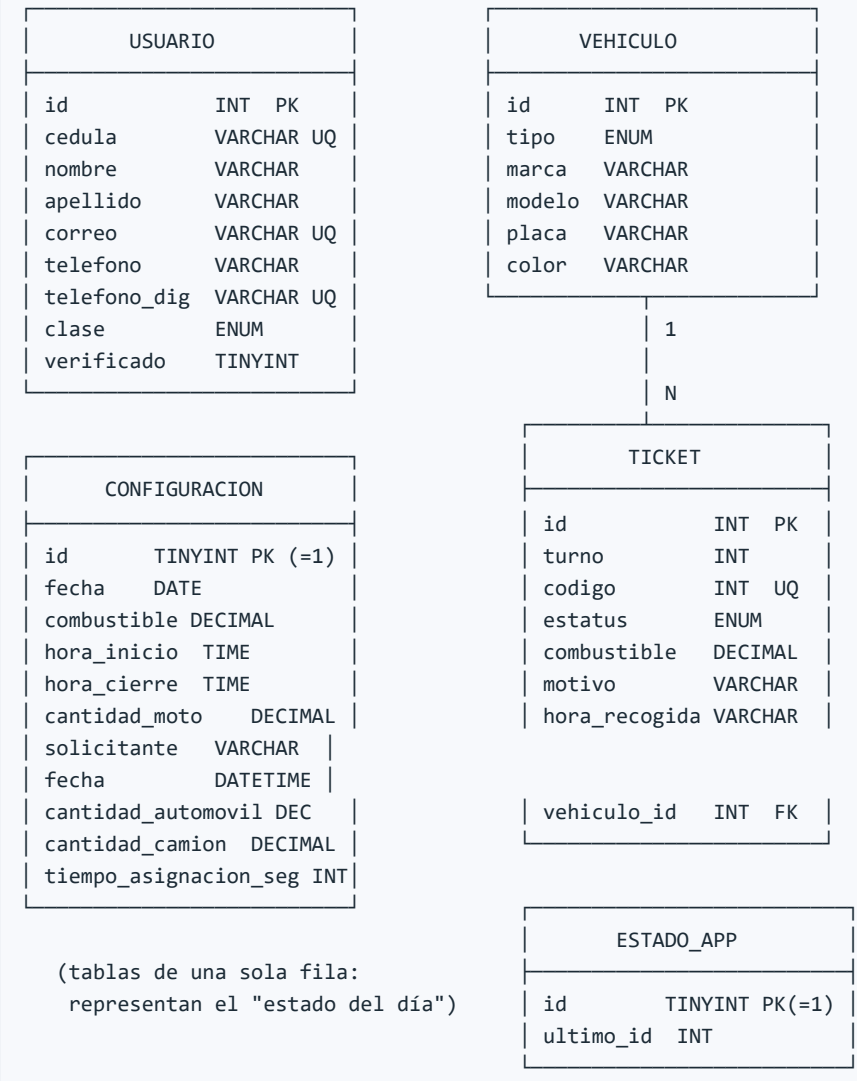
Esta es la parte nueva del sistema y el corazón del documento. Todo lo que se ve en pantalla es una proyección de estas cinco tablas.

5.1 Por qué una base relacional

Se eligió un **modelo relacional normalizado** en lugar de guardar un único JSON en una tabla. Ventajas:

- **Integridad:** la base impide cédulas, correos, teléfonos y códigos de ticket duplicados. La aplicación también lo valida, pero la base es la última línea de defensa.
- **Consultas:** se puede preguntar "cuántos litros por tipo de vehículo" con un `JOIN` y un `GROUP BY`, sin recorrer todo en JavaScript.
- **Relaciones reales:** un ticket *pertenece a* un vehículo mediante una clave foránea, con borrado en cascada.
- **Escalabilidad:** agregar reportes nuevos no obliga a reescribir la capa de datos.

5.2 Diagrama entidad-relación



Relaciones

Relación	Tipo	Implementación
`ticket` → `vehiculo`	N : 1 (muchos tickets por vehículo)	`vehiculo_id` FK con `ON DELETE CASCADE`
`usuario` — `ticket`	No hay FK	El ticket guarda un **identificador de solicitante** (`C-<cédula>` o `P-<token>`), no una FK: el cliente público es anónimo
`configuracion` / `estado_app`	Fila única	`id` fijo en `1`

> **Decisión de diseño.** `ticket` guarda una *copia* de los datos del vehículo en su propia fila de `vehiculo`. Cada ticket crea su vehículo, así que si mañana se edita la marca de un vehículo, los tickets viejos no cambian. Es una **desnormalización intencional** para conservar el histórico.

5.3 Tabla `usuario`

Cuentas del sistema con sus tres roles.

Columna	Tipo	Restricción	Para qué
`id`	`INT AUTO_INCREMENT`	PK	Identificador interno
`cedula`	`VARCHAR(20)`	NOT NULL, **UNIQUE**	Documento de identidad
`nombre`	`VARCHAR(80)`	NOT NULL	Nombre
`apellido`	`VARCHAR(80)`	NOT NULL	Apellido
`correo`	`VARCHAR(150)`	NOT NULL, **UNIQUE**	Login y envío de códigos
`telefono`	`VARCHAR(30)`	NOT NULL	Teléfono tal como lo escribió el usuario
`telefono_digitos`	`VARCHAR(20)`	NOT NULL, **UNIQUE**	Teléfono normalizado (solo dígitos)
`clase`	`ENUM(...)`	NOT NULL	`Administrador` , `Empleado` , `Cliente`
`verificado`	`TINYINT(1)`	NOT NULL, default 0	Solo las verificadas pueden entrar

El truco de `telefono_digitos` . El usuario escribe el teléfono como quiera: `0412-1234567` , `0412 123 4567` , `(0412) 1234567` . Si se guardara tal cual, la misma persona podría registrarse dos veces. Por eso el sistema guarda **dos versiones**: `telefono` (para mostrar) y `telefono_digitos` (solo números) con restricción `UNIQUE` . Así `0412-1234567` y `04121234567` colisionan y la base rechaza el duplicado.

El `ENUM` refleja la jerarquía de clases de JavaScript:

Valor en la base	Clase JS	`rol()`	Permisos
`Administrador`	`Administrador extends Usuario`	`"administrador"`	Todos
`Empleado`	`Empleado extends Usuario`	`"empleado"`	Todos menos Usuarios
`Cliente`	`Cliente extends Usuario`	`"cliente"`	Solo mi cupo de gasolina

5.4 Tabla `vehiculo`

Columna	Tipo	Restricción	Para qué
`id`	`INT AUTO_INCREMENT`	PK	Identificador
`tipo`	`ENUM('moto','automovil','camion')`	NOT NULL	Determina los litros
`marca`	`VARCHAR(80)`	NOT NULL	Ej. Toyota
`modelo`	`VARCHAR(80)`	NOT NULL	Ej. Corolla 2020
`placa`	`VARCHAR(10)`	NOT NULL	Ej. AB123CD
`color`	`VARCHAR(60)`	NOT NULL	Ej. Rojo

El `ENUM` de `tipo` es la contraparte en la base de la jerarquía `Vehiculo → Moto / Automovil / Camion`. Solo admite los tres valores que la fábrica sabe construir.

5.5 Tabla `ticket`

Columna	Tipo	Restricción	Para qué
`id`	`INT`	**PK** (no auto)	Lo asigna la aplicación (`ultimoId + 1`)
`turno`	`INT`	NOT NULL	1–20, único *por día* (se controla en lógica)
`codigo`	`INT`	NOT NULL, **UNIQUE**	1–999, único *global*
`estatus`	`ENUM('pendiente','aceptado','cancelado')`	NOT NULL	Ciclo de vida
`combustible`	`DECIMAL(10,2)`	NULL	Litros solicitados por el cliente (y despachados al aceptar)
`motivo`	`VARCHAR(255)`	NOT NULL, default `''`	Razón de la cancelación
`hora_recogida`	`VARCHAR(5)`	NOT NULL, default `''`	Hora fija `HH:MM` calculada al solicitar
`solicitante`	`VARCHAR(40)`	NOT NULL, default `''`, indexado	`C-<cédula>` o `P-<token>` del cliente
`fecha`	`DATETIME`	NOT NULL	Fecha y hora **en UTC**
`vehiculo_id`	`INT`	NOT NULL, **FK**	Vehículo del ticket

Detalles que vale la pena estudiar:

- **`id` no es `AUTO_INCREMENT`** porque la aplicación ya lleva su propio contador (`estado_app.ultimo_id`). La base solo lo almacena.
- **`codigo` es `UNIQUE` global**, no por día: es un código de verificación que no debe repetirse nunca.
- **`turno` no es `UNIQUE`** porque el turno se reutiliza al día siguiente. La unicidad "por día" se resuelve en la lógica (`turnosUsados()` filtra por fecha) y no se puede expresar con un `UNIQUE` simple.
- **`combustible` guarda los litros solicitados**: el cliente los elige y quedan en el ticket desde que envía la solicitud. Al despachar se descuentan del stock; en tickets viejos o cancelados puede ser `NULL` .

- `hora_recogida` **se calcula una sola vez** al crear la solicitud. Aunque después cambie el horario de la estación, el cliente conserva la hora que se le comunicó.
- `solicitante` permite que cada cliente consulte solo su cupo. Los registrados usan su cédula; el acceso público usa un token aleatorio guardado en su navegador. No es una FK porque el cliente público no tiene fila en `usuario`.
- `DECIMAL(10,2)` en vez de `FLOAT`: los litros se muestran al usuario y `FLOAT` acumula errores de redondeo (0.1 + 0.2 ≠ 0.3).

5.6 Tabla `configuracion`

Una **única fila** (`id = 1`) con el estado del día.

Columna	Tipo	Para qué
`id`	`TINYINT PK` default 1	Fuerza una sola fila
`fecha`	`DATE`	Día al que corresponde la configuración
`combustible`	`DECIMAL(10,2)`	Litros disponibles
`hora_inicio` / `hora_cierre`	`TIME`	Horario de atención
`cantidad_moto`	`DECIMAL(10,2)`	Máximo de litros por moto
`cantidad_automovil`	`DECIMAL(10,2)`	Máximo de litros por automóvil
`cantidad_camion`	`DECIMAL(10,2)`	Máximo de litros por camión
`tiempo_asignacion_seg`	`INT`	Campo heredado del temporizador (ya no se usa en el flujo 2.2; se conserva por compatibilidad)

Por qué una sola fila. `id` es la clave primaria y está fijada en `1`: no se puede insertar una segunda configuración. Es el patrón *singleton* trasladado a la base. La alternativa sería una tabla `configuracion_historica` con una fila por día; se descartó porque la aplicación solo necesita el día en curso.

5.7 Tabla `estado_app`

Columna	Tipo	Para qué
`id`	`TINYINT PK` default 1	Fila única
`ultimo_id`	`INT` default 0	Último `id` de ticket entregado

Es el contador de tickets. Se guarda en la base para que los ids no se repitan al reiniciar el navegador.

> **Detalle de estudio.** La tabla `usuario` usa `AUTO_INCREMENT`, así que sus ids **crecen en cada guardado** (el guardado reemplaza todas las filas). Eso no rompe nada porque ningún ticket apunta a un usuario. En cambio `vehiculo.id` sí se referencia, pero también se recrea completo dentro de la misma transacción.

5.8 Claves, índices y restricciones

Objeto	Tipo	Tabla	Para qué
`PRIMARY`	PK	todas	Identificar la fila
`uq_usuario_cedula`	UNIQUE	usuario	Sin cédulas repetidas
`uq_usuario_correo`	UNIQUE	usuario	Sin correos repetidos
`uq_usuario_telefono`	UNIQUE	usuario	Sin teléfonos repetidos (normalizados)
`uq_ticket_codigo`	UNIQUE	ticket	Código de verificación irrepetible
`idx_ticket_fecha`	INDEX	ticket	Acelera los reportes por fecha
`fk_ticket_vehiculo`	FK	ticket	Integridad referencial + borrado en cascada

Diferencia entre UNIQUE e INDEX. Un `INDEX` solo acelera las búsquedas; un `UNIQUE` acelera **y además prohíbe repetidos**. Una tabla puede tener muchos índices, pero las claves foráneas exigen que la columna referenciada esté indexada (por eso `vehiculo.id`, que es PK, sirve).

`ON DELETE CASCADE`. Si se borra un vehículo, sus tickets se borran solos. En este proyecto el guardado borra primero los tickets y después los vehículos justamente para respetar la jerarquía; aun así la cascada protege ante un borrado manual desde phpMyAdmin.

5.9 Tipos de datos: decisiones y por qué

Necesidad	Tipo elegido	Alternativa descartada	Motivo
Texto corto	`VARCHAR(n)`	`TEXT`	`VARCHAR` reserva solo lo necesario y se indexa mejor
Conjunto cerrado	`ENUM`	`VARCHAR` + `CHECK`	`ENUM` ocupa 1–2 bytes y valida solo
Dinero/litros	`DECIMAL(10,2)`	`FLOAT` / `DOUBLE`	Exactitud decimal; sin errores de redondeo
Fecha y hora	`DATETIME`	`TIMESTAMP`	`TIMESTAMP` tiene rango limitado (1970–2038) y cambia con la zona horaria
Solo hora	`TIME`	`VARCHAR(5)`	Permite comparar y ordenar correctamente
Booleano	`TINYINT(1)`	`BOOLEAN`	En MySQL `BOOLEAN` es un alias de `TINYINT(1)`
Fila única	PK fija en `1`	`LIMIT 1`	La base misma impide duplicados

5.10 Codificación y zona horaria

```
CREATE DATABASE orichuna
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

- **`utf8mb4`** (y no `utf8`) porque el `utf8` de MySQL solo admite 3 bytes y **no puede guardar emojis** ni algunos caracteres. `utf8mb4` es UTF-8 real. Con esto, `María`, `López` y `Pérez` se guardan y se devuelven byte a byte iguales.
- **`utf8mb4_unicode_ci`** define cómo se comparan y ordenan los textos; `ci` = *case insensitive*, así que `MARIA@X.COM` y `maria@x.com` se consideran iguales para el `UNIQUE` del correo.

- **UTC.** PHP fija `date_default_timezone_set('UTC')` y MySQL guarda `DATETIME` en UTC. La API devuelve las fechas como ISO-8601 (`2026-11-03T14:05:00.000Z`) y el navegador las convierte a la hora local (UTC−4 en Venezuela). Así el mismo dato se ve correcto en cualquier país.

6. SQL: creación, consultas y mantenimiento

6.1 Script de creación (`api/schema.sql`)

El script es **idempotente**: usa `IF NOT EXISTS`, así que se puede ejecutar muchas veces sin borrar datos.

```
CREATE DATABASE IF NOT EXISTS orichuna
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE orichuna;

CREATE TABLE IF NOT EXISTS usuario (
  id                INT AUTO_INCREMENT PRIMARY KEY,
  cedula            VARCHAR(20)  NOT NULL,
  nombre            VARCHAR(80)  NOT NULL,
  apellido          VARCHAR(80)  NOT NULL,
  correo            VARCHAR(150) NOT NULL,
  telefono          VARCHAR(30)  NOT NULL,
  telefono_digitos  VARCHAR(20)  NOT NULL,
  clase             ENUM('Administrador','Empleado','Cliente') NOT NULL,
  verificado        TINYINT(1)   NOT NULL DEFAULT 0,
  UNIQUE KEY uq_usuario_cedula   (cedula),
  UNIQUE KEY uq_usuario_correo   (correo),
  UNIQUE KEY uq_usuario_telefono (telefono_digitos)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS vehiculo (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  tipo    ENUM('moto','automovil','camion') NOT NULL,
  marca   VARCHAR(80) NOT NULL,
  modelo  VARCHAR(80) NOT NULL,
  placa   VARCHAR(10) NOT NULL,
  color   VARCHAR(60) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ticket (
  id          INT PRIMARY KEY,
  turno       INT NOT NULL,
  codigo      INT NOT NULL,
  estatus     ENUM('pendiente','aceptado','cancelado') NOT NULL,
  combustible DECIMAL(10,2) NULL,
  motivo      VARCHAR(255) NOT NULL DEFAULT '',
  hora_recogida VARCHAR(5) NOT NULL DEFAULT '',
  solicitante   VARCHAR(40) NOT NULL DEFAULT '',
  fecha       DATETIME NOT NULL,
  vehiculo_id INT NOT NULL,
  UNIQUE KEY uq_ticket_codigo (codigo),
  KEY idx_ticket_fecha (fecha),
  KEY idx_ticket_solicitante (solicitante),
  CONSTRAINT fk_ticket_vehiculo
    FOREIGN KEY (vehiculo_id) REFERENCES vehiculo(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS configuracion (
  id                    TINYINT PRIMARY KEY DEFAULT 1,
  fecha                 DATE NOT NULL,
  combustible           DECIMAL(10,2) NOT NULL,
  hora_inicio           TIME NOT NULL,
```

```
  hora_cierre           TIME NOT NULL,
  cantidad_moto         DECIMAL(10,2) NOT NULL,
  cantidad_automovil    DECIMAL(10,2) NOT NULL,
  cantidad_camion       DECIMAL(10,2) NOT NULL,
  tiempo_asignacion_seg INT NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS estado_app (
  id        TINYINT PRIMARY KEY DEFAULT 1,
  ultimo_id INT NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Qué significa cada cláusula

Cláusula	Significado
`IF NOT EXISTS`	Si ya existe, no hace nada (no borra datos)
`ENGINE=InnoDB`	Motor con transacciones y claves foráneas (MyISAM no las soporta)
`AUTO_INCREMENT`	La base asigna el siguiente número sola
`UNIQUE KEY nombre (columna)`	Índice + prohibición de duplicados
`KEY nombre (columna)`	Índice simple (solo acelera)
`CONSTRAINT nombre FOREIGN KEY`	Declara una relación obligatoria
`ON DELETE CASCADE`	Al borrar el padre, borra los hijos
`DEFAULT 0`	Valor si no se envía la columna

6.2 Consultas de estudio

Estas consultas se pueden pegar en la pestaña **SQL** de phpMyAdmin (`http://localhost/phpmyadmin`) o en consola.

a) Ver todo el estado como lo ve la API

```
SELECT * FROM configuracion;
SELECT * FROM estado_app;
SELECT * FROM usuario ORDER BY id;
SELECT t.id, t.turno, t.codigo, t.estatus, t.combustible, t.fecha,
       v.tipo, v.marca, v.modelo, v.placa, v.color
  FROM ticket t
  JOIN vehiculo v ON v.id = t.vehiculo_id
 ORDER BY t.id;
```

`JOIN` une las dos tablas por la clave foránea: el ticket solo guarda `vehiculo_id`, y la `JOIN` trae los datos del vehículo.

b) Litros entregados por tipo de vehículo (lo que muestra el módulo Reportes)

```
SELECT v.tipo,
       COUNT(*)               AS tickets,
       SUM(t.combustible)     AS litros
  FROM ticket t
  JOIN vehiculo v ON v.id = t.vehiculo_id
 WHERE t.estatus = 'aceptado'
 GROUP BY v.tipo
 ORDER BY litros DESC;
```

c) Litros por marca

```
SELECT v.marca,
       COUNT(*)           AS tickets,
       SUM(t.combustible) AS litros
  FROM ticket t
  JOIN vehiculo v ON v.id = t.vehiculo_id
 WHERE t.estatus = 'aceptado'
 GROUP BY v.marca
 ORDER BY litros DESC;
```

d) Tickets cancelados y su motivo

```
SELECT t.codigo, v.placa, t.motivo, t.fecha
  FROM ticket t
  JOIN vehiculo v ON v.id = t.vehiculo_id
 WHERE t.estatus = 'cancelado'
 ORDER BY t.fecha DESC;
```

e) Turnos usados hoy

```
SELECT turno, COUNT(*) AS veces
  FROM ticket
 WHERE DATE(fecha) = UTC_DATE()
 GROUP BY turno
 ORDER BY turno;
```

f) Buscar un ticket por código de verificación

```
SELECT t.*, v.marca, v.modelo, v.placa
  FROM ticket t
  JOIN vehiculo v ON v.id = t.vehiculo_id
 WHERE t.codigo = 407;
```

g) Combustible restante y porcentaje consumido

```
SELECT c.combustible AS restante,
       1000 - c.combustible AS consumido,
       ROUND((1000 - c.combustible) / 1000 * 100, 1) AS porcentaje
  FROM configuracion c
 WHERE c.id = 1;
```

h) Usuarios por rol

```
SELECT clase, COUNT(*) AS cuantos, SUM(verificado) AS verificados
  FROM usuario
 GROUP BY clase;
```

i) Verificar que no hay duplicados (debe devolver 0 filas)

```
SELECT codigo, COUNT(*) FROM ticket GROUP BY codigo HAVING COUNT(*) > 1;
SELECT correo, COUNT(*) FROM usuario GROUP BY correo HAVING COUNT(*) > 1;
```

j) Tickets por día

```
SELECT DATE(fecha) AS dia, COUNT(*) AS tickets, SUM(combustible) AS litros
  FROM ticket
 GROUP BY DATE(fecha)
 ORDER BY dia DESC;
```

6.3 Operaciones de mantenimiento

Respaldar la base (desde `C:\xampp\mysql\bin`):

```
mysqldump -u root orichuna > respaldo_orichuna.sql
```

Restaurar un respaldo:

```
mysql -u root orichuna < respaldo_orichuna.sql
```

Recrear desde cero (borra todo):

```
DROP DATABASE orichuna;
```

y volver a abrir `api/instalar.php`.

Vaciar solo los datos, conservando tablas:

```
USE orichuna;
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE ticket;
TRUNCATE TABLE vehiculo;
TRUNCATE TABLE usuario;
TRUNCATE TABLE configuracion;
TRUNCATE TABLE estado_app;
SET FOREIGN_KEY_CHECKS = 1;
```

`TRUNCATE` reinicia los contadores `AUTO_INCREMENT`; `DELETE` no.

> **Atajo de la aplicación.** El botón "Restablecer datos de demostración" hace `DELETE api/estado.php`, que equivale a vaciar las cinco tablas. Al recargar, la aplicación detecta que no hay estado y vuelve a sembrar las dos cuentas de ejemplo.

7. La API en PHP

7.1 Endpoints

Método	URL	Cuerpo	Respuesta	Efecto
`GET`	`api/estado.php`	—	JSON del estado o `null`	Leer todo
`POST`	`api/estado.php`	JSON del estado	`{"ok":true}`	Reemplazar todo
`DELETE`	`api/estado.php`	—	`{"ok":true}`	Vaciar todo
`GET`	`api/instalar.php`	—	Texto plano	Crear base y tablas

Cualquier otro método responde `405 Method Not Allowed` con la cabecera `Allow`.

7.2 El contrato JSON

El JSON que viaja es **exactamente** el que produce `EstacionServicio.datosParaGuardar()`. Ese acuerdo entre el frontend y la API se llama *contrato*: si cambia una clave, hay que cambiar los dos lados.


```
{
  "ultimoId": 3,
  "config": {
    "fecha": "2026-11-03",
    "combustible": 998,
    "horaInicio": "07:00",
    "horaCierre": "15:00",
    "cantidadMoto": 2,
    "cantidadAutomovil": 5,
    "cantidadCamion": 15,
    "tiempoAsignacionSeg": 180
  },
  "usuarios": [
    {
      "clase": "Administrador",
      "cedula": "V-12345678",
      "nombre": "Administrador",
      "apellido": "Principal",
      "correo": "rondonsebastian81@gmail.com",
      "telefono": "0412-0000000",
      "verificado": true
    }
  ],
  "tickets": [
    {
      "id": 2,
      "turno": 8,
      "codigo": 661,
      "estatus": "aceptado",
      "combustible": 2,
      "motivo": "",
      "horaRecogida": "09:36",
      "solicitante": "C-V-12345678",
      "fecha": "2026-11-03T13:04:51.000Z",
      "vehiculo": {
        "tipo": "moto",
        "marca": "Honda",
```

```
        "modelo": "CB 190",
        "placa": "XY789Z",
        "color": "Negro"
      }
    }
  ]
}
```

Correspondencia JSON ↔ columnas

JSON	Tabla.columna	Conversión en `POST`
`config.fecha`	`configuracion.fecha`	texto `YYYY-MM-DD`
`config.combustible`	`configuracion.combustible`	`(float)`
`config.horaInicio`	`configuracion.hora_inicio`	`TIME`
`config.tiempoAsignacionSeg`	`configuracion.tiempo_asignacion_seg`	`(int)`
`ultimoId`	`estado_app.ultimo_id`	`(int)`
`usuarios[].clase`	`usuario.clase`	validado contra el `ENUM`
`usuarios[].telefono`	`usuario.telefono` + `telefono_digitos`	se derivan los dígitos
`tickets[].id/turno/codigo`	`ticket.*`	`(int)`
`tickets[].combustible`	`ticket.combustible`	`(float)` o `NULL`
`tickets[].horaRecogida`	`ticket.hora_recogida`	`HH:MM` validada (vacía permitida)
`tickets[].solicitante`	`ticket.solicitante`	texto recortado a 40
`tickets[].fecha`	`ticket.fecha`	ISO-8601 → `DATETIME` UTC
`tickets[].vehiculo`	una fila de `vehiculo`	se inserta y se guarda su `id`

7.3 `api/config.php` — configuración

```
declare(strict_types=1);

const DB_HOST    = '127.0.0.1';
const DB_PORT    = 3306;
const DB_NOMBRE  = 'orichuna';
const DB_USUARIO = 'root';
const DB_CLAVE   = '';   // En XAMPP lo normal es dejarla vacía
```

- `declare(strict_types=1)` obliga a que los tipos declarados se respeten (por ejemplo, no acepta un `"5"` donde se espera `int`).
- Se usan **constantes** y no variables sueltas: no se pueden reasignar por accidente.
- Este archivo **nunca** se sirve al navegador: PHP lo ejecuta en el servidor. Si alguien escribe `http://localhost/orichuna/api/config.php`, ve una página vacía porque no imprime nada.

7.4 `api/db.php` — conexión PDO

```
function pdo(): PDO
{
    static $pdo = null;              // se guarda entre llamadas
    if ($pdo instanceof PDO) return $pdo;

    $dsn = sprintf(
        'mysql:host=%s;port=%d;dbname=%s;charset=utf8mb4',
        DB_HOST, DB_PORT, DB_NOMBRE
    );

    $pdo = new PDO($dsn, DB_USUARIO, DB_CLAVE, [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);

    return $pdo;
}
```

Conceptos clave:

Elemento	Para qué
`static $pdo`	La conexión se crea **una sola vez** por petición (patrón singleton)
`charset=utf8mb4` en el DSN	Sin esto, los acentos llegarían corruptos
`ERRMODE_EXCEPTION`	Un error de SQL lanza excepción en vez de fallar en silencio
`FETCH_ASSOC`	Las filas llegan como arreglos asociativos (`$fila['nombre']`)
`EMULATE_PREPARES = false`	Las sentencias preparadas son **reales** en el servidor, no simuladas por el driver

7.5 `api/estado.php` — lectura (GET)

```
function leer_estado(PDO $pdo): ?array
{
    $cfg = $pdo->query(
        "SELECT DATE_FORMAT(fecha, '%Y-%m-%d')       AS fecha,
                combustible,
                TIME_FORMAT(hora_inicio, '%H:%i')    AS hora_inicio,
                TIME_FORMAT(hora_cierre, '%H:%i')    AS hora_cierre,
                cantidad_moto, cantidad_automovil, cantidad_camion,
                tiempo_asignacion_seg
           FROM configuracion
          WHERE id = 1"
    )->fetch();

    if (!$cfg) return null;   // todavía no hay estado guardado
    // ... consulta de contador, usuarios y tickets ...
}
```

Puntos de estudio:

- **`DATE_FORMAT` y `TIME_FORMAT`** devuelven exactamente el formato que espera el JavaScript (`2026-11-03` y `07:00`). Así no hay que reformatear en JS.

- **Si no hay fila, devuelve `null`.** El JavaScript interpreta `null` como "base vacía" y siembra los datos de demostración.
- `?array` es un tipo de retorno anulable (`array` o `null`).

Y la parte de los tickets:

```
$tickets = $pdo->query(
    "SELECT t.id, t.turno, t.codigo, t.estatus, t.combustible, t.motivo,
            DATE_FORMAT(t.fecha, '%Y-%m-%dT%H:%i:%s.000Z') AS fecha,
            v.tipo, v.marca, v.modelo, v.placa, v.color
       FROM ticket t
       JOIN vehiculo v ON v.id = t.vehiculo_id
      ORDER BY t.id"
)->fetchAll();

return [
    'ultimoId' => $contador ? (int) $contador['ultimo_id'] : 0,
    'config'   => [
        'combustible'  => (float) $cfg['combustible'],
        // ...
    ],
    'usuarios' => array_map(static function (array $u): array {
        return [
            'clase'      => $u['clase'],
            'verificado' => (bool) $u['verificado'],
            // ...
        ];
    }, $usuarios),
    // ...
];
```

Por qué los casteos `(int)`, `(float)`, `(bool)`. PDO devuelve **todo como texto** salvo que se configure lo contrario. Si `combustible` viajara como `"998.00"`, el JavaScript lo trataría como texto y una comparación `combustible <= 0` fallaría. Los casteos garantizan los tipos del contrato.

7.6 `api/estado.php` — escritura (POST)

```
function guardar_estado(PDO $pdo, array $datos): void
{
    $cfg      = $datos['config'] ?? null;
    $usuarios = is_array($datos['usuarios'] ?? null) ? $datos['usuarios'] : [];
    $tickets  = is_array($datos['tickets']  ?? null) ? $datos['tickets']  : [];

    // Validación del servidor: si algo no cumple, responde 422 y NO
    // se abre la transacción (la base queda intacta).
    $combustible = numero_estado($cfg['combustible'] ?? null, 'combustible', 1, 1000000);
    $horaInicio  = hora_estado($cfg['horaInicio'] ?? '', 'horaInicio');
    $horaCierre  = hora_estado($cfg['horaCierre'] ?? '', 'horaCierre');
    if ($horaInicio >= $horaCierre) {
        fallar('La hora de inicio debe ser anterior a la hora de cierre.', 422);
    }
    $cantidadMoto = numero_estado($cfg['cantidadMoto'] ?? null, 'cantidadMoto', 0.5, 1000);
    // ... automóvil, camión, tiempo entero 10–3600 y ultimoId ≥ 0

    $pdo->beginTransaction();
    try {
        // Se reemplaza el estado completo: los volúmenes son pequeños.
        $pdo->exec('DELETE FROM ticket');
```

```
        $pdo->exec('DELETE FROM vehiculo');
        $pdo->exec('DELETE FROM usuario');

        // Configuración (fila única)
        $pdo->prepare(
            'INSERT INTO configuracion (...) VALUES (1, ?, ?, ?, ?, ?, ?, ?, ?)
             ON DUPLICATE KEY UPDATE fecha = VALUES(fecha), ...'
        )->execute([...]);

        // Contador
        $pdo->prepare(
            'INSERT INTO estado_app (id, ultimo_id) VALUES (1, ?)
             ON DUPLICATE KEY UPDATE ultimo_id = VALUES(ultimo_id)'
        )->execute([$ultimoId]);

        // Usuarios, vehículos y tickets (textos recortados al largo de la columna)
        foreach ($usuarios as $u) { /* INSERT INTO usuario ... */ }
        foreach ($tickets  as $t) { /* INSERT vehiculo → último id → INSERT ticket */ }

        $pdo->commit();
    } catch (PDOException $e) {
        if ($pdo->inTransaction()) $pdo->rollBack();
        if ($e->getCode() === '23000') {          // cédula/correo/teléfono/código repetido
            fallar('Hay datos duplicados.', 409);
        }
        throw $e;
    }
}
```

Cuatro ideas fundamentales

1. **Transacción.** `beginTransaction` + `commit` hacen que las diez sentencias sean una sola operación: o se aplican todas, o ninguna. Si falta la luz a mitad de camino, `rollBack()` deja la base como estaba. Sin transacción podrías quedar con tickets sin vehículo.
2. **Reemplazo completo.** Se borra y se vuelve a insertar todo. Es aceptable porque el volumen es mínimo (decenas de filas) y elimina la necesidad de calcular diferencias. La alternativa sería un `UPDATE` por fila comparando qué cambió.
3. `ON DUPLICATE KEY UPDATE` **.** Es un "insertar o actualizar" (`UPSERT`): si la fila con `id = 1` ya existe, la actualiza; si no, la crea. Así funciona tanto la primera vez como las siguientes.
4. **Sentencias preparadas.** `prepare()` + `execute([...])` separan el SQL de los datos. Si un usuario escribe `'; DROP TABLE ticket; --` en su nombre, ese texto se guarda como texto y **no se ejecuta**. Es la defensa contra **inyección SQL**.

> **La API no confía en el navegador.** Aunque la interfaz ya valide, `estado.php` repite las comprobaciones antes de abrir la transacción: números dentro de rango (el combustible admite 0), horas reales, orden del horario, turnos 1–20, códigos 1–999, hora de recogida válida y textos recortados al tamaño de su columna. Un dato inválido produce **HTTP 422** y no modifica nada. Un duplicado produce **HTTP 409** gracias al código SQLSTATE `23000` de la restricción `UNIQUE` . Esto cubre el caso de alguien que llame la API directamente, sin pasar por el formulario.

7.7 `api/estado.php` — borrado (DELETE)

```
function limpiar(PDO $pdo): void
{
    $pdo->beginTransaction();
    try {
        $pdo->exec('DELETE FROM ticket');
        $pdo->exec('DELETE FROM vehiculo');
        $pdo->exec('DELETE FROM usuario');
        $pdo->exec('DELETE FROM configuracion');
        $pdo->exec('DELETE FROM estado_app');
        $pdo->commit();
    } catch (Throwable $e) {
        if ($pdo->inTransaction()) $pdo->rollBack();
        throw $e;
    }
}
```

El orden importa: primero los **hijos** (`ticket`) y después los **padres** (`vehiculo`), para no violar la clave foránea.

7.8 `api/instalar.php` — el instalador

```
$sql = (string) file_get_contents(__DIR__ . '/schema.sql');

// Se descartan las líneas de comentario antes de partir por ';'.
$lineas = [];
foreach (preg_split('/\R/', $sql) ?: [] as $linea) {
    $t = trim($linea);
    if ($t === '' || str_starts_with($t, '--')) continue;
    $lineas[] = $linea;
}

foreach (explode(';', implode("\n", $lineas)) as $sentencia) {
    if (trim($sentencia) === '') continue;
    $pdo->exec($sentencia);
    $ejecutadas++;
}
```

Como `PDO::exec()` ejecuta **una** sentencia por llamada, el instalador parte el archivo `.sql` por `;` . Antes elimina comentarios para no confundir el corte con un `;` dentro de un comentario. Por eso informa `7 sentencias` : 1 `CREATE DATABASE` + 1 `USE` + 5 `CREATE TABLE` .

7.9 Seguridad de la API

Riesgo	Mitigación aplicada
Inyección SQL	Sentencias preparadas con parámetros `?`
Credenciales expuestas	Viven en `config.php`, solo en el servidor
Cuerpo gigante	Límite de 5 MB antes de decodificar
JSON inválido	`json_decode` + comprobación de `is_array`
Números negativos, ceros o fuera de rango	`numero_estado()` en PHP → HTTP 422 antes de abrir la transacción
Horas imposibles u horario invertido	`hora_estado()` + comparación `horaInicio < horaCierre` → HTTP 422
Textos más largos que la columna	`texto_recortado()` a 20/30/60/80/150/255 caracteres
Datos duplicados	Restricciones `UNIQUE` + `PDOException` código `23000` → HTTP 409
Valores de conjuntos cerrados	`ENUM` en la base + validación `in_array` en PHP
Errores con detalles internos	Se registran con `error_log()` y se devuelve un mensaje genérico
Caché de respuestas viejas	Cabecera `Cache-Control: no-store`

Lo que la API no hace (y conviene saber para una defensa). No hay autenticación de API: cualquiera que abra `api/estado.php` desde la misma red puede leer y escribir. Para un proyecto académico local es aceptable; en producción haría falta sesión o token, HTTPS y permisos por rol también en el servidor. El login actual es de interfaz: el servidor no sabe quién es el usuario.

8. Cambios en el frontend

8.1 La clase `Almacenamiento`

Es la **única** pieza que se reescribió. Mantiene la misma interfaz (`cargar`, `guardar`, `limpiar`), así que ninguna vista tuvo que cambiar.

```
class Almacenamiento {
  constructor(clave, almacen, apiUrl) {
    this.clave  = clave;
    this.almacen = almacen || (typeof localStorage !== "undefined" ? localStorage : null);
    this.apiUrl  = apiUrl || "api/estado.php";
    this.modoRemoto = false;   // ¿la API respondió?
    this.ultimoError = null;
  }

  async _peticion(metodo, cuerpo) {          // helper para POST/DELETE
    const opciones = { method: metodo, headers: { "Accept": "application/json" } };
    if (cuerpo !== undefined) {
      opciones.headers["Content-Type"] = "application/json";
      opciones.body = JSON.stringify(cuerpo);
    }
    try {
      const respuesta = await fetch(this.apiUrl, opciones);
      if (!respuesta.ok) return { ok: false, error: "HTTP " + respuesta.status };
      return { ok: true, respuesta };
    } catch (e) {
      return { ok: false, error: String(e && e.message ? e.message : e) };
    }
  }

  async cargar() {
    if (typeof fetch === "function") {
      try {
        const respuesta = await fetch(this.apiUrl, {
          method: "GET", headers: { "Accept": "application/json" }
        });
        if (respuesta.ok) {
          this.modoRemoto = true;
          this.ultimoError = null;
          const datos = await respuesta.json();
          return datos ? EstacionServicio.desdeDatos(datos) : null;
        }
        this.ultimoError = "HTTP " + respuesta.status;
      } catch (e) {
        this.ultimoError = String(e && e.message ? e.message : e);
      }
    }
    this.modoRemoto = false;
    return this._cargarLocal();            // respaldo
  }

  guardar(estacion) {
    if (!estacion) return;
    const datos = estacion.datosParaGuardar();
    this._guardarLocal(datos);             // siempre guarda copia local
    if (!this.modoRemoto) return;          // sin API: ya está guardado
    this._peticion("POST", datos).then((r) => {
      if (!r.ok) console.error("[ORICHUNA] No se pudo guardar en MySQL:", r.error);
```

```
    });
  }

  async limpiar() {
    if (this.modoRemoto) await this._peticion("DELETE");
    if (this.almacen) this.almacen.removeItem(this.clave);
  }
}
```

Por qué `cargar()` es `async` y `guardar()` no

- `cargar()` **necesita** el dato antes de continuar: sin estación no se puede pintar el login. Por eso es `async` y el núcleo hace `await`.
- `guardar()` se llama desde doce puntos distintos del código (vistas y núcleo). Si fuera `async`, habría que poner `await` en todos y volver `async` también esas funciones. Como la interfaz ya está actualizada en memoria, no tiene sentido esperar: se envía la petición y se sigue.

Por qué `modoRemoto`

Si la primera carga usó la API, los guardados se envían al servidor. Si la API no respondió (por ejemplo, abriste el archivo con `file://`), no tiene sentido intentar un `POST` que fallará en cada acción: se queda en modo local.

Por qué siempre se escribe también en `localStorage`

Es una copia de seguridad gratuita. Si el servidor se cae después de arrancar, el estado sigue estando en el navegador. El costo es un `JSON.stringify` de pocos kilobytes.

8.2 `js/nucleo.js`: arranque asíncrono

```
async function iniciar() {
  try {
    estado.estacion = await almacenamiento.cargar();   // ← ahora espera
  } catch (e) {
    estado.estacion = null;
  }

  if (!estado.estacion) {                    // base vacía → datos de demostración
    estado.estacion = new EstacionServicio();
    const admin = new Administrador("V-12345678", "Administrador", "Principal",
                                    "rondonsebastian81@gmail.com", "0412-0000000");
    admin.verificado = true;
    estado.estacion.agregarUsuario(admin);
    // ... el empleado ...
    guardar();                               // se siembra en MySQL
  }

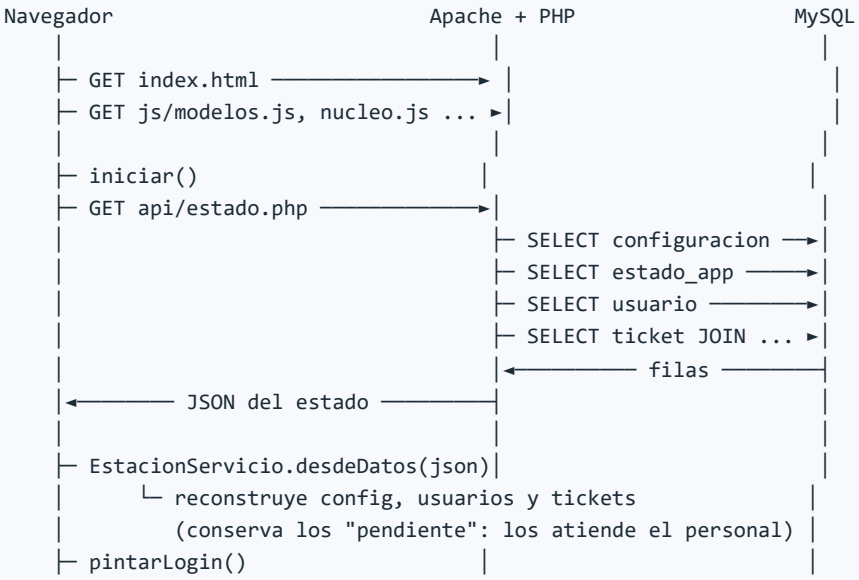
  configurarApp();
  pintarLogin();
}

async function reiniciarDemo() {
  if (!confirm("Se borrarán todos los datos y se volverá al estado inicial. ¿Continuar?")) return;
  await almacenamiento.limpiar();            // ← ahora espera al DELETE
  location.reload();
}
```

Cambios puntuales respecto de la versión 1:

Antes	Ahora	Motivo
`estado.estacion = almacenamiento.cargar()`	`await almacenamiento.cargar()`	La lectura es remota
`function iniciar()`	`async function iniciar()`	Contiene un `await`
`almacenamiento.limpiar(); location.reload()`	`await ... ; location.reload()`	Si no, se recargaba antes del `DELETE`

8.3 Flujo completo de arranque



8.4 Qué pasa si la API falla

Escenario	`cargar()`	`guardar()`	Resultado para el usuario
Todo normal	Datos de MySQL	`POST` a MySQL	Persistencia real
Apache apagado	Intenta, falla, lee local	Solo local	La app funciona; datos en el navegador
`index.html` por `file://`	Fallo de CORS/ruta, lee local	Solo local	Modo demostración
Base vacía	`null` → siembra demo	`POST` con la semilla	Cuentas demo en MySQL
MySQL caído, Apache arriba	`GET` responde 500 → lee local	`POST` falla, se registra en consola	La app sigue usable

9. Programación orientada a objetos

`js/modelos.js` no usa `document` ni `window`: es POO pura. Recibe datos, valida y devuelve resultados.

9.1 `Validador` — métodos estáticos

Cada método devuelve `""` si el dato es válido, o un mensaje de error.

```
static normalizarCedula(texto) {
  // "v12345678", "V 12345678" y "V-12345678" → "V-12345678"
  const base = String(texto).trim().toUpperCase().replace(/[\s.]/g, "");
  const m = /^([VEJ])?-?(\d{6,9})$/.exec(base);
  return m ? (m[1] ? m[1] + "-" : "") + m[2] : base;
}

static validarNumero(valor, campo, opciones) {
  const texto = String(valor === null || valor === undefined ? "" : valor).trim();
  if (texto === "") return campoObligatorio(campo);
  if (!/^-?\d+(\.\d+)?$/.test(texto))       // sin letras ni símbolos
    return campo + " debe ser un número válido.";
  const n = Number(texto), o = opciones || {};
  if (o.entero && !Number.isInteger(n))     return campo + " debe ser un número entero.";
  if (o.positivo && !(n > 0))               return campo + " debe ser un número mayor que 0.";
  if (o.min !== undefined && n < o.min)     return campo + " debe ser mayor o igual que " + o.min + ".";
  if (o.max !== undefined && n > o.max)     return campo + " no puede ser mayor que " + o.max + ".";
  if (o.decimales !== undefined &&
      (texto.split(".")[1] || "").length > o.decimales)
    return campo + " admite como máximo " + o.decimales + " decimales.";
  return "";                                  // vacío = válido
}
```

Los métodos son `static` porque **no necesitan estado**: no dependen de un objeto concreto. Se llaman `Validador.validarPlaca("AB123CD")`.

Reglas aplicadas en todos los formularios

Campo	Regla
Cédula	Letra opcional (V/E/J) + 6 a 9 dígitos; se normaliza a `V-12345678`; máximo 20 caracteres
Nombre y apellido	Solo letras, espacios, apóstrofo, punto y guion; máximo 80 caracteres
Marca y color	Solo letras; máximo 80 y 60 caracteres
Modelo	Letras, números y los signos `+ - . '`; máximo 80 caracteres
Correo	Formato `usuario@dominio.ext`, sin puntos mal ubicados; máximo 150 caracteres
Teléfono	De 7 a 15 dígitos; admite `+`, espacios, paréntesis y guion; máximo 30 caracteres
Placa	De 6 a 8 alfanuméricos, **con al menos una letra y un número**; sin espacios ni guiones
Combustible del día	De 0 a 1 000 000 L, hasta 2 decimales (0 = sin litros disponibles)
Máximo de litros por vehículo	De 0,5 a 1000 L, hasta 2 decimales
Litros solicitados por el cliente	Mayor que 0 y menor o igual al máximo de su tipo de vehículo
Hora de recogida	`HH:MM` calculada una vez al solicitar (reparto de 20 turnos en el horario)
Horas de inicio y cierre	`HH:MM` entre 00:00 y 23:59; el inicio debe ser estrictamente anterior al cierre
Motivo de cancelación	Texto libre recortado a 255 caracteres
Búsqueda	Código de exactamente 3 dígitos o placa de 3 a 8 alfanuméricos con letras y números

Los mensajes para campos vacíos se generan con `campoObligatorio(campo)`, que quita el artículo y evita errores de género ("El campo «cantidad de combustible» es obligatorio").

9.1.1 Validar antes de modificar: el error de los números negativos

La primera versión de `guardarEstacion()` escribía los valores del formulario directamente en la configuración **y después** llamaba a `validar()`:

```
// ❌ versión con el error: primero muta, luego valida
estado.estacion.config.combustible = Number(input.value);   // el -5 ya quedó en memoria
if (errores.length) return;                                  // el aviso llega tarde
```

Si la validación fallaba, el valor negativo **ya estaba en memoria**. El siguiente `guardar()` —crear un ticket, cancelar, editar un usuario— enviaba ese `-5` a la API y a la base de datos. El campo se veía rechazado en pantalla, pero el dato inválido se filtraba en la siguiente operación.

La corrección valida sobre un **borrador** y solo copia a la configuración real si no hay errores:

```
const borrador = new ConfiguracionDistribucion();
Object.assign(borrador, leerValores());
const errores = borrador.validar();
if (!listarErrores(errores, ORC.$("c-errores"))) return;   // no se toca nada
borrador.normalizar();
Object.assign(estado.estacion.config, borrador);           // recién ahora
guardar();
```

La misma idea se aplica a usuarios, vehículos y registro de clientes: el objeto se construye, se valida y solo entonces se agrega a la estación. Además, `api/estado.php` repite las reglas y responde **HTTP 422** si el JSON trae un negativo, un decimal fuera de rango, una hora imposible o un texto que no cabe en la columna; si hay

cédula, correo, teléfono o código repetido responde **HTTP 409**. Así, aunque alguien llame la API directamente con `curl`, la base no acepta el dato.

9.2 `Usuario` y la herencia

```
class Usuario {                        // clase base (padre)
  constructor(cedula, nombre, apellido, correo, telefono) {
    this.cedula = String(cedula).trim();
    this.verificado = false;           // no puede entrar hasta verificarse
  }
  validar() {
    const errores = [];
    errores.push(Validador.validarCedula(this.cedula));
    // ... resto de campos ...
    return errores.filter(Boolean);    // quita las cadenas vacías
  }
  rol()         { return "usuario"; }
  puede(accion) { return false; }
}

class Administrador extends Usuario {
  rol()         { return "administrador"; }
  puede(accion) { return true; }
}

class Empleado extends Usuario {
  rol()         { return "empleado"; }
  puede(accion) { return accion !== "gestion_usuarios"; }
}

class Cliente extends Usuario {
  rol()         { return "cliente"; }
  puede(accion) { return accion === "asignar"; }
}
```

Polimorfismo. El mismo mensaje `usuario.puede("reportes")` se envía a objetos de clases distintas y cada uno responde distinto. El núcleo no pregunta "¿qué tipo eres?": envía el mensaje y el objeto decide. Eso es exactamente lo que se guarda en `usuario.clase` y en el `ENUM` de la base.

9.3 `Vehiculo`, subtipos y el patrón Fábrica

```
class Vehiculo {
  constructor(tipo, marca, modelo, placa, color) {
    this.tipo  = tipo;
    this.placa = String(placa).trim().toUpperCase();
  }
  obtenerCombustible(config) { return 0; }   // implementación por defecto
}

class Moto extends Vehiculo {
  constructor(marca, modelo, placa, color) { super("moto", marca, modelo, placa, color); }
  obtenerCombustible(config) { return config.cantidadMoto; }        // 2 L
}

class FabricaVehiculos {
  static crear(tipo, marca, modelo, placa, color) {
    switch (tipo) {
      case "moto":      return new Moto(marca, modelo, placa, color);
```

```
      case "automovil": return new Automovil(marca, modelo, placa, color);
      case "camion":    return new Camion(marca, modelo, placa, color);
      default: throw new Error("Tipo de vehículo desconocido: " + tipo);
    }
  }
}
```

El formulario envía el tipo como texto (`"moto"` , `"automovil"` , `"camion"`); la fábrica decide qué clase instanciar. El mismo `tipo` se guarda en el `ENUM` de `vehiculo.tipo` .

9.4 `ConfiguracionDistribucion` — el día

```
horarioAMinutos(hora) {
  const partes = String(hora).split(":").map(Number);   // "07:00" → [7, 0]
  return partes[0] * 60 + partes[1];                    // → 420 minutos
}

enHorario() {
  const ahora = this.minutosActuales();                 // hora actual en minutos
  return ahora >= this.horarioAMinutos(this.horaInicio) &&
         ahora <  this.horarioAMinutos(this.horaCierre);
}

cantidadPara(vehiculo) {
  return vehiculo.obtenerCombustible(this);             // delega en el polimorfismo
}
```

Convertir `"HH:MM"` a minutos permite comparar horas sin fechas ni zonas horarias.

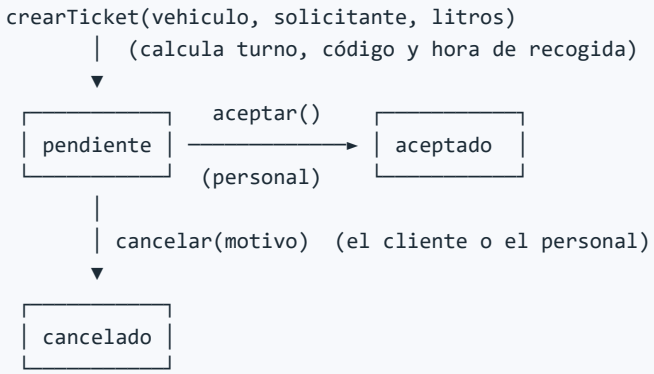
```
validar() {
  const errores = [];
  errores.push(Validador.validarNumero(this.combustible, "La cantidad de combustible",
    { min: 0, max: 1000000, decimales: 2 }));      // 0 L = sin combustible
  errores.push(Validador.validarNumero(this.cantidadMoto, "El litraje de moto",
    { positivo: true, min: 0.5, max: 1000, decimales: 2 }));
  // ... automóvil y camión con las mismas reglas
  errores.push(Validador.validarHora(this.horaInicio, "La hora de inicio"));
  errores.push(Validador.validarHora(this.horaCierre, "La hora de cierre"));
  const limpios = errores.filter(Boolean);
  if (limpios.length === 0 &&
      this.horarioAMinutos(this.horaInicio) >= this.horarioAMinutos(this.horaCierre)) {
    limpios.push("La hora de inicio debe ser anterior a la hora de cierre.");
  }
  return limpios;
}

normalizar() {                    // solo después de validar sin errores
  ["combustible", "cantidadMoto", "cantidadAutomovil",
   "cantidadCamion", "tiempoAsignacionSeg"].forEach((clave) => {
    this[clave] = Number(this[clave]);
  });
  return this;
}
```

`validar()` reúne los errores de los siete campos (rango, decimales, tipo entero, horas y orden del horario) y `normalizar()` convierte los textos del formulario a números. La configuración se rechaza completa si un solo

campo falla: nunca queda a medias. `crearTicket()` vuelve a llamar a `validar()` como defensa, de modo que no se emiten tickets con una configuración inválida.

9.5 `Ticket` — ciclo de vida




```
// El cliente elige los litros; el personal confirma el despacho.
aceptar(estacion) {
  if (this.estatus === "aceptado") return { error: "Esta solicitud ya fue despachada." };
  if (this.estatus !== "pendiente") return { error: "Esta solicitud ya está cancelada." };
  const litros = Number(this.combustible);
  const maximo = Number(estacion.config.cantidadPara(this.vehiculo));
  if (!Number.isFinite(litros) || litros <= 0 || litros > maximo) {
    return { error: "La solicitud no tiene una cantidad de litros válida." };
  }
  if (!estacion.config.hayCombustible()) {       // stock en 0
    return { error: "No hay litros disponibles en la estación en este momento." };
  }
  if (Number(estacion.config.combustible) < litros) {
    return { error: "No hay litros disponibles suficientes para despachar esta solicitud." };
  }
  estacion.config.combustible -= litros;          // descuenta del día
  this.estatus = "aceptado";
  this.motivo = "";
  return { ok: true };
}

cancelar(motivo) {
  if (this.estatus === "cancelado") return;
  this.estatus = "cancelado";
  this.motivo  = motivo;
}
```

A diferencia de la versión 2.1, `aceptar()` **no cancela** el ticket cuando falta combustible: devuelve un error y la solicitud sigue pendiente para que el personal reponga combustible o la cancele. Los tres valores de `this.estatus` siguen siendo exactamente los del `ENUM` de `ticket.estatus`.

9.6 `EstacionServicio` — las reglas

a) Turnos y códigos únicos

```
turnosUsados() {
  const hoy = this.fechaActual();
  const usados = new Set();
  this.tickets.forEach(t => {
```

```
    if (aISO(t.fecha) === hoy && t.estatus !== "cancelado") usados.add(t.turno);
  });
  return usados;
}

turnoDisponible() {
  const usado  = this.turnosUsados();
  const libres = [];
  for (let n = 1; n <= 20; n++) if (!usado.has(n)) libres.push(n);
  if (libres.length === 0) return null;                       // 20/20
  return libres[Math.floor(Math.random() * libres.length)];   // uno al azar
}

codigoDisponible() {
  const usado  = new Set(this.tickets.map(t => t.codigo));
  const libres = [];
  for (let n = 1; n <= 999; n++) if (!usado.has(n)) libres.push(n);
  return libres[Math.floor(Math.random() * libres.length)];
}
```

Un `Set` guarda cada valor una sola vez, así que `has(n)` responde "¿está ocupado?" en tiempo constante. Se construye la lista de libres y se escoge una al azar. Los turnos de solicitudes canceladas se liberan.

a.1) Hora de recogida

Los 20 turnos se reparten uniformemente entre la hora de inicio y la de cierre. Con `07:00–15:00` (480 minutos), cada turno cae cada 24 minutos: el 1 a las `07:00` y el 20 a las `14:36`.

```
minutoRecogida(turno) {
  const inicio = this.horarioAMinutos(this.horaInicio);
  const cierre = this.horarioAMinutos(this.horaCierre);
  const duracion = Math.max(1, cierre - inicio);
  const n = Math.min(20, Math.max(1, Math.floor(Number(turno)) || 1));
  return inicio + Math.floor(((n - 1) * duracion) / 20);
}

horaRecogidaPara(turno) {
  const minuto = this.minutoRecogida(turno) % (24 * 60);
  return String(Math.floor(minuto / 60)).padStart(2, "0") + ":" +
    String(minuto % 60).padStart(2, "0");
}
```

`crearTicket()` guarda esa hora en el ticket. Si luego se cambia el horario de la estación, los tickets ya emitidos conservan su hora original.

b) Login por correo o teléfono

```
buscarUsuarioPorIdentificador(texto) {
  const t = String(texto).trim().toLowerCase();
  return this.usuarios.find(u => {
    if (!u.verificado) return false;          // solo cuentas verificadas
    return u.correo.toLowerCase() === t ||
           digitos(u.telefono) === digitos(t);
  });
}

function digitos(texto) {
  return String(texto).replace(/[^\d]/g, ""); // "0412-1234567" → "04121234567"
}
```

`digitos()` es la misma idea que `telefono_digitos` en la base: normalizar para comparar.

c) Búsqueda y reportes

```
buscar(texto) {
  const valor = String(texto).trim();
  if (!valor) return [];
  if (/^\d{3}$/.test(valor))                      // 3 dígitos → por código
    return this.tickets.filter(t => t.codigo === Number(valor));
  const pl = valor.toUpperCase();                 // si no → por placa
  return this.tickets.filter(t => t.vehiculo.placa.includes(pl));
}
```

9.7 Persistencia y reconstrucción

```
datosParaGuardar() {
  return {
    ultimoId: this.ultimoId,
    config: { fecha, combustible, horaInicio, horaCierre,
              cantidadMoto, cantidadAutomovil, cantidadCamion, tiempoAsignacionSeg },
    usuarios: this.usuarios.map(u => ({
      clase: u.constructor.name,        // "Administrador" | "Empleado" | "Cliente"
      cedula, nombre, apellido, correo, telefono, verificado
    })),
    tickets: this.tickets.map(t => ({
      id, turno, codigo, estatus, combustible, motivo,
      fecha: t.fecha.toISOString(),
      vehiculo: { tipo, marca, modelo, placa, color }
    }))
  };
}

static desdeDatos(d) {
  const es = new EstacionServicio();
  if (d.config) Object.assign(es.config, d.config);
  (d.usuarios || []).forEach(u => {
    const nuevo = u.clase === "Administrador" ? new Administrador(...)
                : u.clase === "Cliente"       ? new Cliente(...)
                : new Empleado(...);
    nuevo.verificado = !!u.verificado;
    es.usuarios.push(nuevo);
  });
  (d.tickets || []).forEach(t => {
    const vehiculo = FabricaVehiculos.crear(t.vehiculo.tipo, ...);
    const ticket   = new Ticket(es, vehiculo, t.id);
    ticket.fecha = new Date(t.fecha);
    ticket.turno = t.turno;  ticket.codigo = t.codigo;
    ticket.estatus = t.estatus; ticket.combustible = t.combustible;
    ticket.motivo = t.motivo;
    es.tickets.push(ticket);
  });
  return es;
}
```

El roundtrip. JSON no puede guardar métodos ni instancias. Al guardar se convierten los objetos a datos planos con el **nombre de la clase** (`u.constructor.name`). Al cargar, `desdeDatos` vuelve a instanciar la clase correcta y reconstruye los vehículos con la fábrica. Sin este paso, `usuario.rol()` no existiría al recargar.

Con la base de datos el roundtrip es **el mismo**, solo cambia el transporte: antes `localStorage.setItem`, ahora un `POST` a PHP. El contrato JSON es idéntico, y por eso la migración no tocó ninguna vista.

10. Flujos completos

10.1 Login de administrador

```
1. escribe correo o teléfono
2. buscarUsuarioPorIdentificador() lo encuentra (solo verificados)
3. abrirModalCodigo() genera 4 dígitos, los muestra y los envía por EmailJS
4. confirmarModal() compara lo escrito con lo generado
5. si coincide → abrirSesion(usuario) → pintarNav() + navegar("inicio")
   si no       → error en el modal sin perder el flujo
```

10.2 Solicitar un cupo (cliente) y despacharlo (personal)

```
asignacion.js (cliente)
  f-enviar → solicitarCupo()
      ├─ FabricaVehiculos.crear(...)         ← polimorfismo
      ├─ vehiculo.validar()                  ← validaciones
      ├─ config.cantidadPara(vehiculo)       ← máximo permitido
      ├─ estacion.crearTicket(vehiculo, solicitante, litros)
      │     ├─ rechaza si no hay litros o si no hay cupos
      │     ├─ rechaza otra solicitud activa del mismo cliente
      │     ├─ crea turno y código únicos
      │     └─ fija horaRecogida
      ├─ guardar()                           ← POST a MySQL
      └─ pintarAsignacion()                  ← tarjeta con turno, hora y estatus

solicitudes.js (administrador/empleado)
  Despachar → despachar(id)
      ├─ ticket.aceptar(estacion)            ← valida y descuenta litros
      ├─ guardar()                           ← POST a MySQL
      └─ pintarSolicitudes()                 ← muestra el resultado

  Cancelar → prompt("Motivo") → ticket.cancelar(motivo) → guardar()
```

El botón **Actualizar lista** (y **Actualizar estado** en la vista del cliente) vuelve a leer el estado del servidor para ver los cambios hechos en otra sesión.

10.3 Recargar la página con una solicitud pendiente

```
1. iniciar() → cargar() → GET api/estado.php
2. desdeDatos() reconstruye la solicitud con estatus "pendiente",
   su horaRecogida y su solicitante
3. la solicitud SIGUE pendiente: ya no se cancela al recargar
4. el cliente la ve al entrar (mismo token/cédula) y el personal la
   encuentra en «Solicitudes de cupo»
```

Antes de la versión 2.2, `finalizarPendientes()` cancelaba los tickets pendientes al recargar porque el temporizador de 3 minutos ya no tenía sentido. Ahora el despacho es manual: la solicitud debe sobrevivir a la recarga hasta que el personal la atienda.

11. Guía de estudio

11.1 Recorrido recomendado (en este orden)

Paso	Qué leer	Qué entender
1	Este documento, secciones 1–4	Qué hace el sistema y cómo se separan las capas
2	Sección 5 (modelo de datos)	Las 5 tablas, claves y relaciones
3	`api/schema.sql`	Cómo se traduce el modelo a SQL
4	Sección 6.2 (consultas)	Cómo se pregunta a la base
5	Sección 7 (API)	Cómo viaja el estado entre JS y MySQL
6	Sección 8 (frontend)	El contrato JSON y el respaldo local
7	Sección 9 (POO)	Herencia, polimorfismo, fábrica
8	Sección 10 (flujos)	Cómo se conecta todo en una acción real

11.2 Preguntas de repaso (con respuesta)

1. ¿Por qué el navegador no puede conectarse directo a MySQL? Porque necesitaría las credenciales en el código del cliente, visibles para cualquiera, y cualquier visitante podría leer o borrar la base. PHP actúa de intermediario y las guarda en el servidor.

2. ¿Qué diferencia hay entre `INDEX` y `UNIQUE`? Un `INDEX` solo acelera las búsquedas. Un `UNIQUE` además prohíbe valores repetidos.

3. ¿Por qué `telefono_digitos` existe si ya está `telefono`? Para comparar sin importar el formato. `0412-1234567` y `04121234567` son el mismo número, y con `UNIQUE` sobre los dígitos la base impide registrarlo dos veces.

4. ¿Por qué `ticket.codigo` es `UNIQUE` y `ticket.turno` no? El código no debe repetirse nunca. El turno se reutiliza cada día (1–20), así que su unicidad es "por día" y se controla en la lógica, no con una restricción simple.

5. ¿Qué pasa si se corta la luz a mitad de un `POST`? Nada: la operación está dentro de una transacción y `rollBack()` deshace lo que se alcanzó a escribir. La base queda como antes.

6. ¿Por qué `combustible` es `DECIMAL` y no `FLOAT`? `FLOAT` no representa exacto los decimales y acumula errores de redondeo. `DECIMAL(10,2)` guarda exactamente dos decimales.

7. ¿Qué hace `ON DUPLICATE KEY UPDATE`? Inserta la fila; si ya existe una con la misma clave primaria, la actualiza. Sirve para las tablas de una sola fila sin preguntar antes si existen.

8. ¿Por qué `cargar()` es `async`? Porque espera la respuesta HTTP. `guardar()` no lo es porque la interfaz ya está actualizada y no necesita esperar.

9. ¿Qué pasa si se abre `index.html` sin Apache? `fetch` falla, `modoRemoto` queda en `false` y todo se guarda en `localStorage`. La app funciona, pero los datos no se comparten entre navegadores.

10. ¿Cómo se evita la inyección SQL? Con sentencias preparadas (`prepare` + `execute` con parámetros `?`). El SQL y los datos viajan por canales separados, así que un dato nunca se interpreta como código.

11. ¿Por qué las fechas se guardan en UTC? Para que el mismo instante se vea correcto en cualquier zona horaria. El navegador convierte UTC a la hora local del usuario.

12. ¿Qué guarda `estado_app.ultimo_id` y por qué no basta `AUTO_INCREMENT`? El id del último ticket. `ticket.id` lo asigna la aplicación y debe coincidir con el que ya conoce el navegador; `AUTO_INCREMENT` lo asignaría la base y el objeto en memoria quedaría desincronizado.

13. ¿Qué es el contrato JSON? El acuerdo sobre la forma exacta del JSON entre `datosParaGuardar()` y `estado.php`. Si cambia una clave en un lado, hay que cambiarla en el otro.

14. ¿Por qué el `POST` borra y vuelve a insertar todo? Porque el volumen es mínimo y así se evita calcular diferencias. Es una decisión de simplicidad, no la única posible; con más datos se haría un `UPDATE` por fila.

15. ¿Qué relación hay entre `Usuario` / `Administrador` / `Empleado` y el `ENUM` de la base? `u.constructor.name` produce exactamente `"Administrador"`, `"Empleado"` o `"Cliente"`, que son los valores permitidos del `ENUM`. La jerarquía de clases se refleja en un solo campo.

11.3 Ejercicios propuestos

1. **Consulta.** Escribe una consulta que devuelva la placa y el código de los tickets cancelados por "Combustible insuficiente".
2. **Índice.** Agrega un índice sobre `vehiculo.placa` y explica qué consulta se beneficia.
3. **Columna nueva.** Agrega `usuario.fecha_registro DATETIME` al esquema y al `INSERT` de la API. ¿Qué habría que cambiar en el JSON?
4. **Endpoint.** Agrega `GET api/estado.php?resumen=1` que devuelva solo los totales (tickets, litros, usuarios) usando `COUNT` y `SUM`.
5. **Transacción.** Fuerza un error a propósito (por ejemplo, un `codigo` repetido) y comprueba con `SELECT` que no quedó nada a medias.
6. **Respaldo.** Genera un `mysqldump`, borra la base, restáurala y verifica que los tickets siguen ahí.
7. **Seguridad.** Explica qué habría que agregar para que un `POST` solo lo pueda hacer un administrador autenticado.
8. **POO.** Agrega la clase `Camioneta extends Vehiculo` con 8 litros. Enumera todos los lugares que habría que tocar (incluido el `ENUM` de la base).

11.4 Glosario

Término	Definición
API	Interfaz que permite a un programa pedirle datos a otro
Endpoint	URL concreta de una API (`api/estado.php`)
JSON	Formato de texto para intercambiar datos
CRUD	Crear, leer, actualizar, borrar
PK (Primary Key)	Columna que identifica de forma única cada fila
FK (Foreign Key)	Columna que apunta a la PK de otra tabla
Índice	Estructura que acelera las búsquedas
Transacción	Grupo de operaciones que se aplican todas o ninguna
Commit / Rollback	Confirmar / deshacer una transacción
UPSERT	Insertar si no existe, actualizar si existe
Sentencia preparada	Consulta con parámetros separados de los datos (evita inyección SQL)
PDO	Capa de PHP para acceder a bases de datos
DSN	Cadena con los datos de conexión (`mysql:host=...`)
Normalización	Organizar los datos en tablas sin redundancia
ENUM	Tipo que solo admite una lista cerrada de valores
utf8mb4	Codificación UTF-8 completa (admite emojis y acentos)
UTC	Hora universal coordinada, sin zona horaria
ISO-8601	Formato de fecha `2026-11-03T14:05:00.000Z`
Polimorfismo	Un mismo mensaje, respuestas distintas según la clase
Herencia	Una clase deriva de otra y reutiliza su comportamiento
Fábrica	Patrón que centraliza la creación de objetos
IIFE	Función que se ejecuta al definirse y aísla variables
Fallback	Mecanismo de respaldo cuando algo falla

12. Anexos

12.1 Inventario de archivos

Archivo	Líneas aprox.	Rol
`index.html`	120	Estructura y puntos de montaje
`css/estilos.css`	500	Diseño
`js/modelos.js`	540	POO + persistencia
`js/nucleo.js`	510	Estado, login, navegación, EmailJS
`js/vistas/*.js`	7 archivos	Una pantalla cada uno
`api/config.php`	15	Credenciales
`api/db.php`	30	Conexión PDO
`api/estado.php`	330	API de estado
`api/instalar.php`	50	Instalador
`api/schema.sql`	95	Esquema

12.2 Correspondencia entre capas

Concepto de negocio	Clase JS	Tabla SQL
Usuario / rol	`Usuario`, `Administrador`, `Empleado`, `Cliente`	`usuario`
Vehículo y tipo	`Vehiculo`, `Moto`, `Automovil`, `Camion`	`vehiculo`
Ticket y estatus	`Ticket`	`ticket`
Configuración del día	`ConfiguracionDistribucion`	`configuracion`
Contador de ids	`EstacionServicio.ultimoId`	`estado_app`
Persistencia	`Almacenamiento`	API `estado.php`

12.3 Lista de verificación para la defensa

- [ ] Explicar las cinco tablas y sus claves.
- [ ] Mostrar la base en phpMyAdmin con datos reales.
- [ ] Ejecutar una consulta con `JOIN` y `GROUP BY` en vivo.
- [ ] Crear un ticket y mostrar cómo cambia la fila en `ticket` y `configuracion.combustible`.
- [ ] Explicar por qué se usa PHP y no JavaScript para conectarse.
- [ ] Explicar la transacción y qué evita.
- [ ] Mostrar el respaldo con `mysqldump`.
- [ ] Explicar herencia, polimorfismo y fábrica en `modelos.js`.
- [ ] Explicar el contrato JSON entre `datosParaGuardar()` y `estado.php`.
- [ ] Mostrar que la app sigue funcionando si se apaga Apache (respaldo local).

12.4 Registro de cambios

Versión	Fecha	Cambios
1.0	2026-09	Versión inicial: HTML/CSS/JS con POO, persistencia en `localStorage`, EmailJS
2.0	2026-11	Base de datos MySQL/MariaDB, API PHP con PDO, respaldo en `localStorage`, corrección del botón "Confirmar asignación", documentación del modelo de datos
2.1	2026-11	Validación completa de todos los formularios, corrección del bug de números negativos en "Surtir estación", validación de servidor con HTTP 422/409, búsqueda estricta y conservación de la verificación al editar usuarios
2.2	2026-11	Flujo cliente-solicita / personal-despacha, hora de recogida repartida en el horario, litros elegidos dentro del máximo, módulo «Solicitudes de cupo», aviso de «no hay litros disponibles» y persistencia de la solicitud al recargar

Qué cambió en la versión 2.1

Área	Antes	Ahora
Combustible y litros	Aceptaba negativos, ceros y decimales libres	Rango 1–1 000 000 L y 0,5–1000 L, máximo 2 decimales
Tiempo de asignación	Cualquier número	Entero entre 10 y 3600 segundos
Horas	Solo comparaba el orden	Formato `HH:MM` real y orden estricto
Cédula / placa	Patrones amplios	Cédula normalizada; placa con letras y números
Correo / teléfono	Patrones básicos	Formato estricto y longitud de dígitos
Textos	Sin límite	Recortados al tamaño de la columna
Duplicados	Mensaje genérico	Mensaje específico (cédula, correo o teléfono)
Editar usuario	Perdía la verificación	Conserva `verificado`
Verificación de cuenta nueva	El modal olvidaba la cuenta pendiente	Se guarda la cuenta y se puede reintentar el código
Búsqueda	Un número de 2 dígitos caía en la búsqueda por placa	Solo código de 3 dígitos o placa con letras y números
API	Guardaba lo que llegara	Rechaza datos inválidos con 422 y duplicados con 409

Qué cambió en la versión 2.2

Área	Antes (2.1)	Ahora (2.2)
Quién asigna	Administrador/empleado asignaba y confirmaba	El **cliente** solicita; el personal **acepta y despacha**
Temporizador	3 minutos y cancelación automática	Eliminado del flujo; la solicitud espera al personal
Litros	Fijos según el tipo de vehículo	El cliente elige, hasta el máximo del tipo
Hora de recogida	No existía	Reparto de los 20 turnos entre inicio y cierre
Visibilidad del cliente	Veía stock y turnos libres	Solo su turno, placa, litros solicitados, estatus y hora
Sin combustible	El ticket se cancelaba al aceptar	Aviso «No hay litros disponibles» y botón deshabilitado
Recargar	Cancelaba los pendientes	La solicitud pendiente se conserva
Combustible en 0 L	Se rechazaba (mínimo 1 L)	Es un estado válido: la estación queda sin litros y el cliente ve el aviso
Módulos	«Asignación de cupo» para todos	«Mi cupo de gasolina» (cliente) y «Solicitudes de cupo» (personal)

Qué cambió respecto de la versión 1.0

Aspecto	1.0	2.0
Persistencia	`localStorage`	MySQL/MariaDB (API PHP)
Transporte	`JSON.stringify` / `JSON.parse`	HTTP + JSON
Instalación	Abrir `index.html`	XAMPP + `api/instalar.php`
Datos compartidos	Solo en ese navegador	En cualquier navegador de la red
Archivos nuevos	—	`api/config.php`, `db.php`, `estado.php`, `instalar.php`, `schema.sql`
Código modificado	—	`Almacenamiento`, `iniciar()`, `reiniciarDemo()`, `asignacion.js`

Qué se conservó intacto

- Toda la lógica de negocio (`EstacionServicio`, `Ticket`, `Usuario`, `Vehiculo`…).
- Todas las vistas: ninguna sabe si los datos vienen de MySQL o del navegador.
- El contrato `datosParaGuardar()` / `desdeDatos()`: el mismo JSON, otro destino.
- El modo demostración sin servidor.

> **Idea para recordar.** El cambio de `localStorage` a MySQL no tocó las reglas del negocio ni las pantallas: solo se sustituyó la pieza que guarda. Eso es posible porque el sistema está diseñado en capas y la persistencia es una frontera única. Ese es el argumento más fuerte del diseño orientado a objetos del proyecto.